Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of

Electrical & Computer Engineering

Lab Report 1

Implementation of Linear and Logistic Regression for Regression and Classification Problems

Course Code: ECE 4246

Course Title: Machine Learning Sessional

Submitted to:
Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:
Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Even

Submission Date: January 26, 2026

Contents

1 Theory and Introduction **1**
1.1 Linear Regression . . . 1
1.1.1 Hypothesis Function . . . 1
1.1.2 Cost Function (Mean Squared Error) . . . 1
1.1.3 Optimization (Gradient Descent) . . . 1
1.2 Logistic Regression . . . 2
1.2.1 Sigmoid Activation and Hypothesis . . . 2
1.2.2 Cost Function (Log Loss) . . . 2
1.2.3 Gradient Descent Update . . . 2

2 Methodology **2**
2.1 Datasets Used . . . 2
2.2 Algorithms Implemented . . . 3
2.2.1 Task 1: Linear Regression (Regression Problem) . . . 3
2.2.2 Task 2: Linear Regression (Classification Problem) . . . 4
2.2.3 Task 3: Logistic Regression (Regression Problem) . . . 5
2.2.4 Task 4: Logistic Regression (Classification Problem) . . . 6

3 Results and Discussion **7**
3.1 Linear Regression Results . . . 7
3.1.1 Task 1: Regression on Continuous Data . . . 7
3.1.2 Task 2: Linear Regression on Classification Data . . . 7
3.2 Logistic Regression Results . . . 8
3.2.1 Task 3: Logistic Regression on Regression Data . . . 8
3.2.2 Task 4: Logistic Regression on Classification Data . . . 9

4 Conclusion **10**

References **10**

Implementation of Linear and Logistic Regression for Regression and Classification Problems

1 Theory and Introduction

Linear Regression and Logistic Regression are fundamental algorithms in supervised machine learning. In this experiment, both algorithms were implemented from first principles using Python (NumPy and Pandas) to analyze their mathematical properties and performance across regression and classification domains.

1.1 Linear Regression

Linear Regression models the relationship between a dependent variable y and one or more independent variables X by fitting a linear equation to observed data [1].

1.1.1 Hypothesis Function

The hypothesis function $h_\theta(x)$ approximates the target value as a weighted sum of the input features:

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \theta^T x \tag{1}$$

where θ represents the parameter vector (weights and bias) and x represents the feature vector.

1.1.2 Cost Function (Mean Squared Error)

To measure the accuracy of the hypothesis, the Mean Squared Error (MSE) cost function $J(\theta)$ is defined as the average of the squared differences between predicted and actual values:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2 \tag{2}$$

where m is the number of training examples. The factor $\frac{1}{2}$ is included to simplify the derivative during gradient calculation.

1.1.3 Optimization (Gradient Descent)

The parameters θ are updated iteratively to minimize $J(\theta)$ using the Gradient Descent algorithm:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \tag{3}$$

Substituting the derivative of the MSE, the update rule becomes:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)} \tag{4}$$

where α is the learning rate.

1.2 Logistic Regression

Logistic Regression is a classification algorithm used to estimate the probability that a given instance belongs to a particular class (e.g., $y = 1$) [2].

1.2.1 Sigmoid Activation and Hypothesis

Unlike Linear Regression, the output is transformed using the Sigmoid (logistic) function $g(z)$ to map predictions to the range $[0, 1]$:

$$g(z) = \frac{1}{1 + e^{-z}} \tag{5}$$

[Image of Sigmoid function graph]

The hypothesis is therefore defined as the probability $P(y = 1|x; \theta)$:

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}} \tag{6}$$

1.2.2 Cost Function (Log Loss)

The MSE function is non-convex for Logistic Regression, which can lead to local minima. Instead, the Log Loss (Binary Cross-Entropy) function is used:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right] \tag{7}$$

This cost function penalizes confident wrong predictions heavily and is convex, ensuring a global minimum.

1.2.3 Gradient Descent Update

The gradient of the Log Loss function is mathematically identical to that of Linear Regression, though the hypothesis $h_\theta(x)$ definition differs:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)} \tag{8}$$

2 Methodology

The implementation was divided into four distinct tasks. For all tasks, the dataset was split into training (80%) and testing (20%) sets. Feature scaling (Standardization) was applied where necessary to ensure convergence of Gradient Descent.

2.1 Datasets Used

Two primary datasets were selected to evaluate the models:

- **Regression Dataset (Housing Data):** The *California Housing* dataset was utilized to test the regression capabilities of the algorithms. This dataset contains metrics such as median income and house age to predict median house values [3].
- **Classification Dataset (Iris):** The standard *Iris* dataset was used for classification tasks. To ensure binary classification, the dataset was filtered to include only two species: *Versicolor* and *Virginica* [4].

2.2 Algorithms Implemented

2.2.1 Task 1: Linear Regression (Regression Problem)

This model was implemented to predict continuous values. The cost function $J(\theta)$ was minimized using Gradient Descent.

Algorithm 1 Linear Regression (Gradient Descent)

Input: X (Features), y (Target), α (Learning Rate), N (Iterations)
Initialize: Weights $\theta \leftarrow \vec{0}$, Bias added to X
for $i = 1$ to N **do**
  $\hat{y} \leftarrow X \cdot \theta$ ▷ Linear Hypothesis
  $error \leftarrow \hat{y} - y$
  $gradient \leftarrow \frac{1}{m} X^T \cdot error$
  $\theta \leftarrow \theta - \alpha \cdot gradient$ ▷ Update Rule
  $J(\theta) \leftarrow \frac{1}{2m} \sum (\hat{y} - y)^2$ ▷ Compute MSE Cost
end for
Output: Optimized θ

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 1. LOAD DATA (Fixed URL)
def load_data():
    url = "https://raw.githubusercontent.com/krishnaik06/Simple-Linear-Regression/master/Salary_Data.csv"

    try:
        df = pd.read_csv(url)
        print("Dataset loaded successfully!")
    except Exception as e:
        print(f"Error loading online data: {e}")
        return None, None

    # X = Features (YearsExperience), y = Target (Salary)
    X = df.iloc[:, :-1].values
    y = df.iloc[:, -1].values
    return X, y

# 2. MATHEMATICAL FUNCTIONS (From Scratch)
def train_test_split_custom(X, y, test_size=0.2, random_state=42):
    np.random.seed(random_state)
    m = len(y)
    indices = np.random.permutation(m)
    test_count = int(m * test_size)

    test_indices = indices[:test_count]
    train_indices = indices[test_count:]

    return X[train_indices], X[test_indices], y[train_indices], y[test_indices]

def compute_cost(X, y, theta):
    m = len(y)
    predictions = X.dot(theta)
    cost = (1 / (2 * m)) * np.sum(np.square(predictions - y))
    return cost

def gradient_descent(X, y, theta, learning_rate, iterations):
    m = len(y)
    cost_history = []

    for i in range(iterations):
        predictions = X.dot(theta)
        errors = predictions - y
        # Gradient = (1/m) * X.T * errors
        gradient = (1 / m) * X.T.dot(errors)
        # Update Rule
        theta -= learning_rate * gradient
        # Track Cost
        cost_history.append(compute_cost(X, y, theta))
    return theta, cost_history

def predict(X, theta):
    return X.dot(theta)

# 3. METRICS (Regression Specific)
def r2_score_custom(y_true, y_pred):
    mean_y = np.mean(y_true)
    ss_total = np.sum((y_true - mean_y) ** 2)
    ss_residual = np.sum((y_true - y_pred) ** 2)
    return 1 - (ss_residual / ss_total)

def mse_custom(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)
```

```python
# 4. MAIN EXECUTION
X, y = load_data()
# Plot Raw Data
plt.figure(figsize=(8, 5))
plt.scatter(X, y, color='blue', label='Data Points')
plt.title('Raw Data: Years Experience vs Salary')
plt.xlabel('Years Experience')
plt.ylabel('Salary')
plt.show()
# B. Preprocessing
X_b = np.c_[np.ones((len(X), 1)), X]
# C. Split Data
X_train, X_test, y_train, y_test = train_test_split_custom(X_b, y, test_size=0.2)

# D. Train Model
theta = np.zeros(X_train.shape[1]) # Init weights
learning_rate = 0.01
iterations = 1000
print(f"\nTraining Linear Regression (LR={learning_rate}, Iter={iterations})...")
theta_final, cost_history = gradient_descent(X_train, y_train, theta, learning_rate, iterations)

# E. Evaluate
y_pred_train = predict(X_train, theta_final)
y_pred_test = predict(X_test, theta_final)

mse_val = mse_custom(y_test, y_pred_test)
r2_val = r2_score_custom(y_test, y_pred_test)

print("\n" + "="*40)
print("EVALUATION REPORT")
print("="*40)
print(f"Final Weights (Theta): {theta_final}")
print(f"Mean Squared Error (MSE): {mse_val:,.2f}")
print(f"R2 Score (Accuracy): {r2_val:.4f}")
print("(Confusion Matrix / ROC are not applicable to Regression)")


# 5. FINAL VISUALIZATION
plt.figure(figsize=(14, 5))
# Plot 1: Cost History (Convergence)
plt.subplot(1, 2, 1)
plt.plot(cost_history, color='red', linewidth=2)
plt.title('Training Cost (Loss) over Iterations')
plt.xlabel('Iterations')
plt.ylabel('Cost (MSE)')
plt.grid(True, alpha=0.3)
# Plot 2: Model Fit Line
plt.subplot(1, 2, 2)
# Scatter Test Data
plt.scatter(X_test[:, 1], y_test, color='green', s=100, label='Test Data (Actual)')
# Plot Regression Line
x_range = np.linspace(X[:,0].min(), X[:,0].max(), 100)
x_range_b = np.c_[np.ones((100, 1)), x_range] # Add intercept for prediction
y_line = predict(x_range_b, theta_final)

plt.plot(x_range, y_line, color='blue', linewidth=3, label='Model Prediction Line')
plt.title('Linear Regression Model Fit')
plt.xlabel('Years Experience')
plt.ylabel('Salary')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

2.2.2 Task 2: Linear Regression (Classification Problem)

Linear Regression was adapted for classification by applying a threshold to the continuous output.

Algorithm 2 Linear Regression for Classification

Train: Run *Linear Regression (Gradient Descent)* to find θ
Predict:
for each sample x in X_{test} **do**
    $\hat{y}_{raw} \leftarrow x \cdot \theta$
    if $\hat{y}_{raw} \geq 0.5$ **then**
        $\hat{y}_{class} \leftarrow 1$
    else
        $\hat{y}_{class} \leftarrow 0$
    end if
end for
Evaluate: Compare $\hat{y}_{class}$ with true labels.

Only the classification-specific code is shown here.

```python
# 2. MATHEMATICAL FUNCTIONS (Linear Regression)

def compute_cost_mse(X, y, theta):
    """
    Cost Function: Mean Squared Error (MSE)
    Even though this is classification, Linear Regression uses MSE.
    """
    m = len(y)
    predictions = X.dot(theta)
    cost = (1/(2*m)) * np.sum(np.square(predictions - y))
    return cost

def gradient_descent(X, y, theta, learning_rate, iterations):
    m = len(y)
    cost_history = []
    for i in range(iterations):
        predictions = X.dot(theta)
        errors = predictions - y
        # Gradient Calculation
        gradient = (1/m) * X.T.dot(errors)
        # Update
        theta -= learning_rate * gradient
        # Log Cost
        cost_history.append(compute_cost_mse(X, y, theta))
    return theta, cost_history

def predict_class(X, theta, threshold=0.5):
    continuous_pred = X.dot(theta)
    return (continuous_pred >= threshold).astype(int), continuous_pred

# 3. EVALUATION METRICS
def confusion_matrix(y_true, y_pred):
    TP = np.sum((y_true == 1) & (y_pred == 1))
    TN = np.sum((y_true == 0) & (y_pred == 0))
    FP = np.sum((y_true == 0) & (y_pred == 1))
    FN = np.sum((y_true == 1) & (y_pred == 0))
    return np.array([[TN, FP], [FN, TP]])

def calculate_metrics(y_true, y_pred):
    cm = confusion_matrix(y_true, y_pred)
    TN, FP, FN, TP = cm.ravel()

    acc = (TP + TN) / (TP + TN + FP + FN)
    prec = TP / (TP + FP) if (TP+FP) > 0 else 0
    rec = TP / (TP + FN) if (TP+FN) > 0 else 0
    f1 = 2 * (prec * rec) / (prec + rec) if (prec+rec) > 0 else 0

    return acc, prec, rec, f1, cm
```

2.2.3 Task 3: Logistic Regression (Regression Problem)

Since Logistic Regression is a classifier, the regression target (e.g., House Price) was converted into a binary target (High vs. Low Value) based on the median price.

Algorithm 3 Logistic Regression (Target Transformation)

Pre-processing:
$median \leftarrow Median(y_{continuous})$
$y_{binary} \leftarrow [1 \text{ if } y_i > median \text{ else } 0 \text{ for } y_i \text{ in } y]$
Train: Run *Logistic Regression* on (X, y_{binary})
Evaluate: Classification metrics (Accuracy, F1) on the binned target.

Only the target transformation code is shown here.

```python
# 2. LOGISTIC REGRESSION MODEL (From Scratch)
def sigmoid(z):
    # Clip to prevent overflow/underflow
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))

def train_logistic_regression(X, y, lr=0.1, epochs=5000):
    # Add Intercept
    X_b = np.c_[np.ones((len(X), 1)), X]
    theta = np.zeros(X_b.shape[1])
    m = len(y)
    cost_history = []

    for i in range(epochs):
        # Forward
        z = np.dot(X_b, theta)
        h = sigmoid(z)
        # Gradient (Derivatives)
        gradient = np.dot(X_b.T, (h - y)) / m
        # Update
        theta -= lr * gradient
        # Cost (Log Loss)
        epsilon = 1e-15
        h = np.clip(h, epsilon, 1 - epsilon) # Safety for log
        cost = -np.mean(y * np.log(h) + (1 - y) * np.log(1 - h))
        cost_history.append(cost)

    return theta, cost_history

def predict_proba(X, theta):
    X_b = np.c_[np.ones((len(X), 1)), X]
    return sigmoid(np.dot(X_b, theta))

def predict(X, theta, threshold=0.5):
    probs = predict_proba(X, theta)
    return (probs >= threshold).astype(int)
```

2.2.4 Task 4: Logistic Regression (Classification Problem)

The standard implementation using the Sigmoid activation function and Log Loss cost.

Algorithm 4 Logistic Regression (Standard)

Input: X, y, α, N
Function: $h_\theta(x) = \frac{1}{1+e^{-X\theta}}$
for $i = 1$ to N **do**
  $z \leftarrow X \cdot \theta$
  $\hat{y} \leftarrow \sigma(z)$ ▷ Sigmoid Activation
  $gradient \leftarrow \frac{1}{m} X^T \cdot (\hat{y} - y)$
  $\theta \leftarrow \theta - \alpha \cdot gradient$
  $Cost \leftarrow -\frac{1}{m} \sum [y \log(\hat{y}) + (1-y)\log(1-\hat{y})]$
end for

```python

def standardize(X):
    # Standardization (Z-score Normalization) is crucial for Logistic Regression
    mean = np.mean(X, axis=0)
    std = np.std(X, axis=0)
    return (X - mean) / std, mean, std

def sigmoid(z):
    # Sigmoid Activation: 1 / (1 + e^-z)
    z = np.clip(z, -500, 500) # prevent overflow
    return 1.0 / (1.0 + np.exp(-z))

def compute_cost(X, y, theta):
    # Log Loss (Binary Cross Entropy)
    m = len(y)
    h = sigmoid(X.dot(theta))
    epsilon = 1e-15 # prevent log(0)
    h = np.clip(h, epsilon, 1 - epsilon)
    cost = (-1/m) * np.sum(y * np.log(h) + (1 - y) * np.log(1 - h))
    return cost

def train_logistic_regression(X, y, lr=0.1, epochs=3000):
    # Add Intercept (Bias)
    X_b = np.c_[np.ones((len(X), 1)), X]
    theta = np.zeros(X_b.shape[1])
    m = len(y)
    cost_history = []

    for _ in range(epochs):
        # Forward Prop
        z = np.dot(X_b, theta)
        h = sigmoid(z)

        # Gradient Calculation
```

```
        gradient = np.dot(X_b.T, (h - y)) / m

        # Update Weights
        theta -= lr * gradient

        # Record Cost
        cost_history.append(compute_cost(X_b, y, theta))

    return theta, cost_history

def predict(X, theta, threshold=0.5):
    X_b = np.c_[np.ones((len(X), 1)), X]
    probs = sigmoid(np.dot(X_b, theta))
    preds = (probs >= threshold).astype(int)
    return preds, probs
```

3 Results and Discussion

3.1 Linear Regression Results

3.1.1 Task 1: Regression on Continuous Data

The Linear Regression model was applied to the regression dataset to predict continuous outcomes. The training process was initialized with a learning rate of 0.01 for 1000 iterations. The model parameters converged to weights of $\theta = [22997.42, 9768.48]$, indicating a bias term of approximately 22,997 and a feature coefficient of 9,768.

```
Training Linear Regression (LR=0.01, Iter=1000)...

========================================
EVALUATION REPORT
========================================
Final Weights (Theta): [22997.42197203  9768.48083188]
Mean Squared Error (MSE): 52,876,610.79
R2 Score (Accuracy):      0.8965
(Confusion Matrix / ROC are not applicable to Regression)
```

An R^2 score of 0.8965 suggests that approximately 89.65% of the variance in the target variable is explained by the model. As observed in Figure 1, the cost function decreased rapidly, and the regression line demonstrates a strong fit to the data points. The console output is as follows:

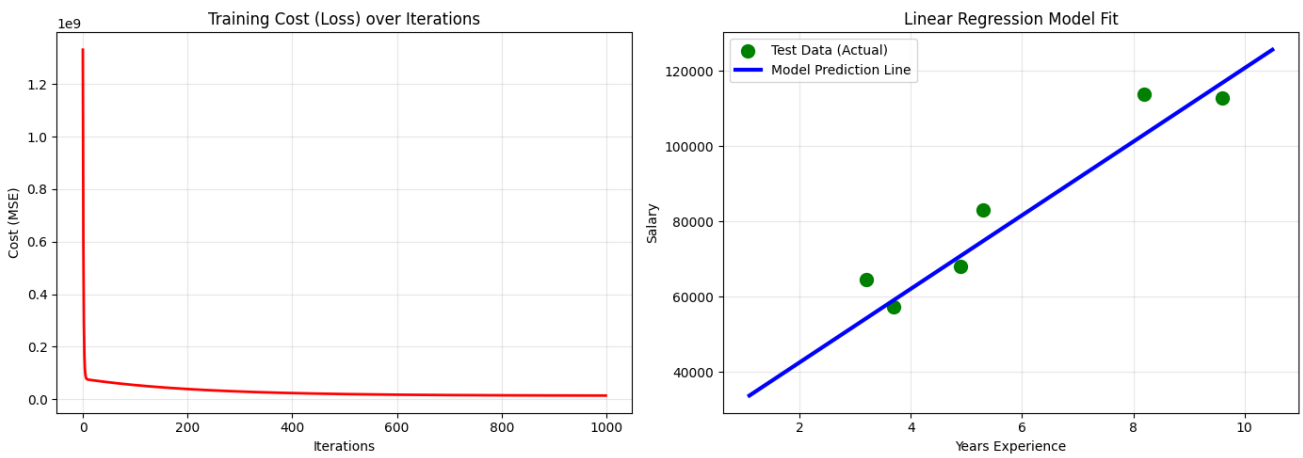

Figure 1: Task 1 Evaluation: Training Cost history (Left) and Linear Model Fit (Right).

3.1.2 Task 2: Linear Regression on Classification Data

The Linear Regression algorithm was experimentally applied to the Iris classification task by using a decision threshold of 0.5 on the predicted continuous values. Despite the algorithm being designed for

regression, it achieved high accuracy on this linearly separable subset of the data. The console output is as follows:

```
Dataset loaded successfully!
Training Linear Regression on Classification Data...

========================================
EVALUATION REPORT
========================================
Algorithm: Linear Regression (MSE) applied to Classification
Threshold: 0.5
----------------------------------------
Accuracy:  0.9500
Precision: 1.0000
Recall:    0.8750
F1 Score:  0.9333

Confusion Matrix:
      Pred 0  Pred 1
Act 0   12       0
Act 1   1       7
```

The confusion matrix indicates that while the precision was perfect (no False Positives), there was one False Negative (Recall of 0.875), resulting in a slight drop in the F1 score. Figure 2 illustrates the ROC curve and the decision boundary established by the linear function.

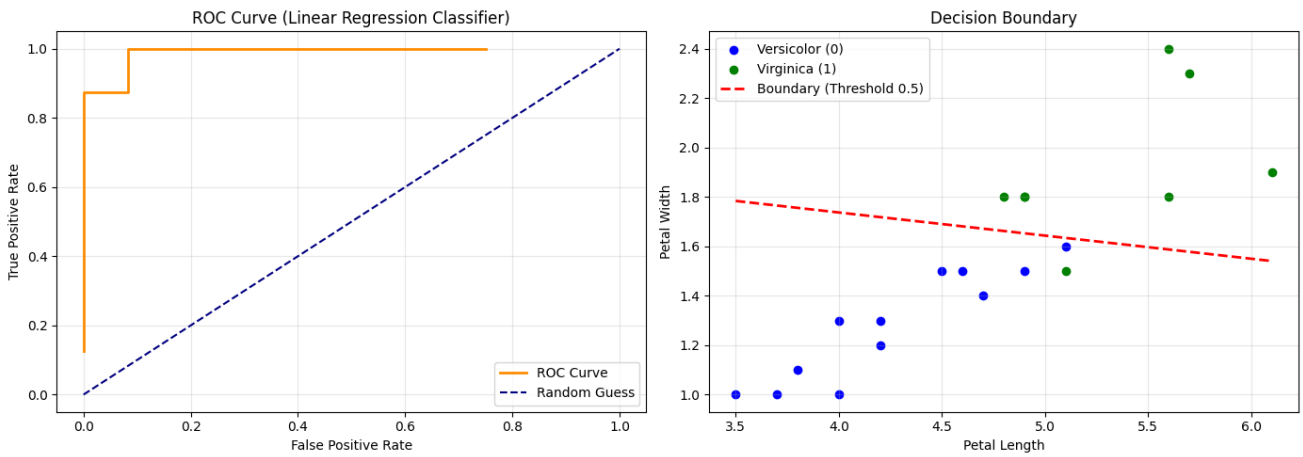

Figure 2: Task 2 Evaluation: ROC Curve (Left) and Decision Boundary (Right).

3.2 Logistic Regression Results

3.2.1 Task 3: Logistic Regression on Regression Data

To apply Logistic Regression to the Housing dataset, the continuous target was transformed into a binary classification problem. The median house price was calculated as $181,600. Houses priced above this median were labeled "High Value," and those below or equal were labeled "Low Value."

```
Median House Price: $181,600
Task: Predict if House Price is > Median (High Value) or <= Median (Low Value)
Training Logistic Regression on Housing Data...

========================================
```

```
EVALUATION REPORT (Housing Data)
========================================
Accuracy:  80.00%
Precision: 79.17%
Recall:    79.17%
F1 Score:  0.7917
--------------------
Confusion Matrix:
[[84 20]
 [20 76]]
```

The model achieved an accuracy of 80%, with the confusion matrix showing a balanced distribution of errors (20 False Positives and 20 False Negatives). Figure 3 displays the cost reduction, the decision boundary separating high and low-value regions, and the resulting confusion matrix.

Figure 3: Task 3 Evaluation: Training Cost (Left), Decision Boundary for High/Low Price (Middle), and Confusion Matrix (Right).

3.2.2 Task 4: Logistic Regression on Classification Data

Finally, the standard Logistic Regression model was tested on the binary Iris dataset (100 samples). The model converged successfully, providing probabilistic classification predictions.

```
Dataset Size: 100 samples (Versicolor & Virginica)
Training Logistic Regression...

========================================
LOGISTIC REGRESSION REPORT
========================================
Accuracy:  90.00%
Precision: 87.50%
Recall:    87.50%
F1 Score:  0.8750
--------------------
Confusion Matrix:
[[11  1]
 [ 1  7]]
```

The confusion matrix reveals two misclassifications (one False Positive and one False Negative) out of the test set, resulting in an accuracy of 90%. Figure 4 provides a comprehensive view of the model's performance, including the cost history, ROC curve, confusion matrix, and the decision boundary relative to the data points.

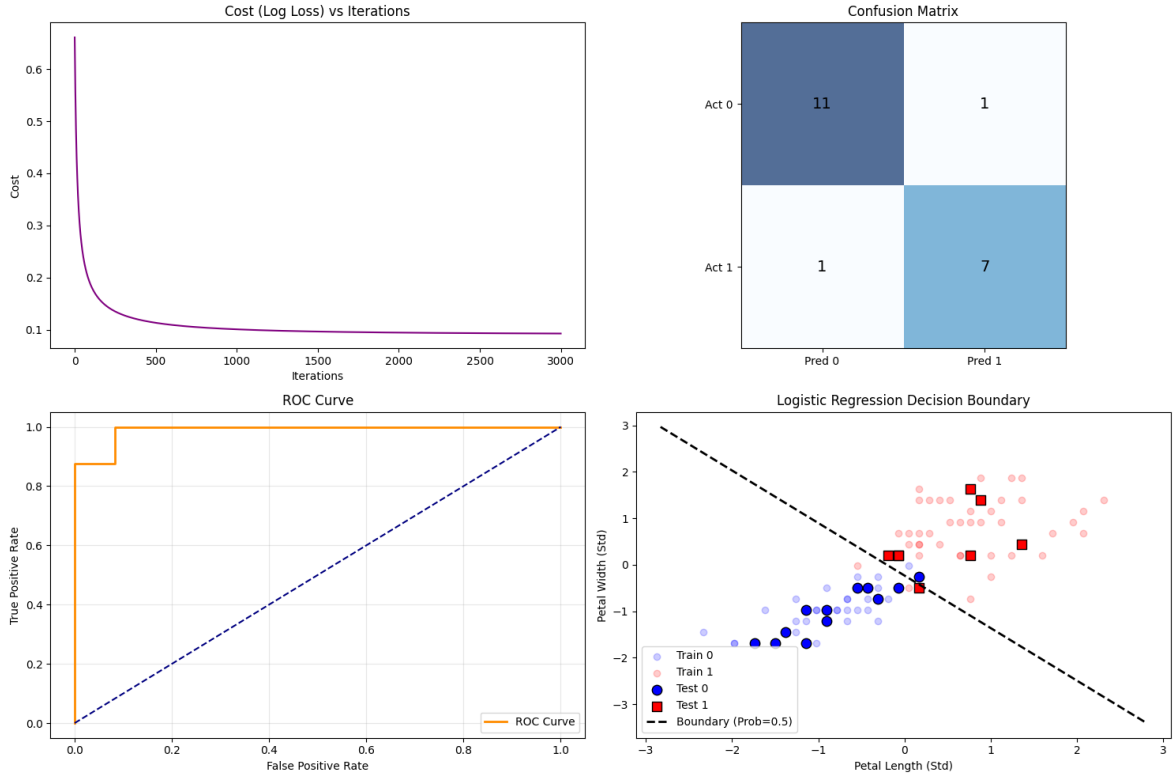

Figure 4: Task 4 Evaluation: Training Cost (Top-Left), Confusion Matrix (Top-Right), ROC Curve (Bottom-Left), and Decision Boundary (Bottom-Right).

4 Conclusion

In this sessional, Linear and Logistic Regression algorithms were implemented from scratch and evaluated across different domains. Linear Regression proved highly effective for the regression task, achieving an R^2 score of 0.8965. When forced into a classification role (Task 2), it performed surprisingly well (95% accuracy) but remains theoretically suboptimal due to its sensitivity to outliers and unbounded output. Logistic Regression demonstrated robust performance on the standard classification task (90% accuracy). When adapted for regression data via binning (Task 3), it provided a viable method for coarse-grained prediction (High vs. Low value), achieving 80% accuracy. The successful convergence of the cost functions in all four tasks confirms the correctness of the Gradient Descent implementation.

References

[1] C. M. Bishop, *Pattern recognition and machine learning.* springer, 2006.

[2] T. Hastie, R. Tibshirani, and J. Friedman, *The elements of statistical learning: data, mining, inference, and prediction.* Springer Science & Business Media, 2009.

[3] R. K. Pace and R. Barry, "Sparse spatial autoregressions," *Statistics & probability letters*, vol. 33, no. 3, pp. 291–297, 1997.

[4] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of eugenics*, vol. 7, no. 2, pp. 179–188, 1936.